

TELEGRAM SUPPORT SYSTEM · SYSTEM ARCHITECTURE

Telegram Technical Support Bot for the PenGuide Platform

This system was designed as an organized support channel between PenGuide users and the administration team. It turns a normal Telegram conversation into a clear, trackable ticket lifecycle while keeping the user experience simple.

Telegram Bot

Support System

System Architecture

AI-Assisted Development

My role in the project

I did not approach the project as traditional manual coding. My role was the **System Architect**: I designed the logic, defined the ticket lifecycle, selected the supporting components, used Prompt Engineering to guide AI-assisted code generation, and managed hosting and operation.

Logic design

Defined how a message moves from the user to administration and how replies are mapped back to the correct ticket owner.

Ticket lifecycle

Designed ticket creation, unique identifiers, replies, closing, and state updates.

Prompt Engineering

Created precise instructions for AI-assisted implementation of the defined system logic.

Hosting & operations

Configured continuous operation and monitoring so the service can stay available.

TECHNICAL FOUNDATION

Technical architecture and operating principles

The system uses simple, direct components for the core job: receive a request, preserve the ticket-to-user relationship, and route each response back to the correct user.

Technical architecture

Runtime	Python 3 hosted on Replit
Bot interface	Telegram Bot API
Data layer	SQLite / TinyDB for identifiers and ticket state

Privacy principle

The system is designed to retain only the information needed for routing and ticket management. The goal is to map a ticket to a user and its state without turning the database into a full archive of message content.

What is retained?

User identifier / Chat ID, Ticket ID, ticket state, and the routing data needed to map an administrator response to the correct user.

What is not required?

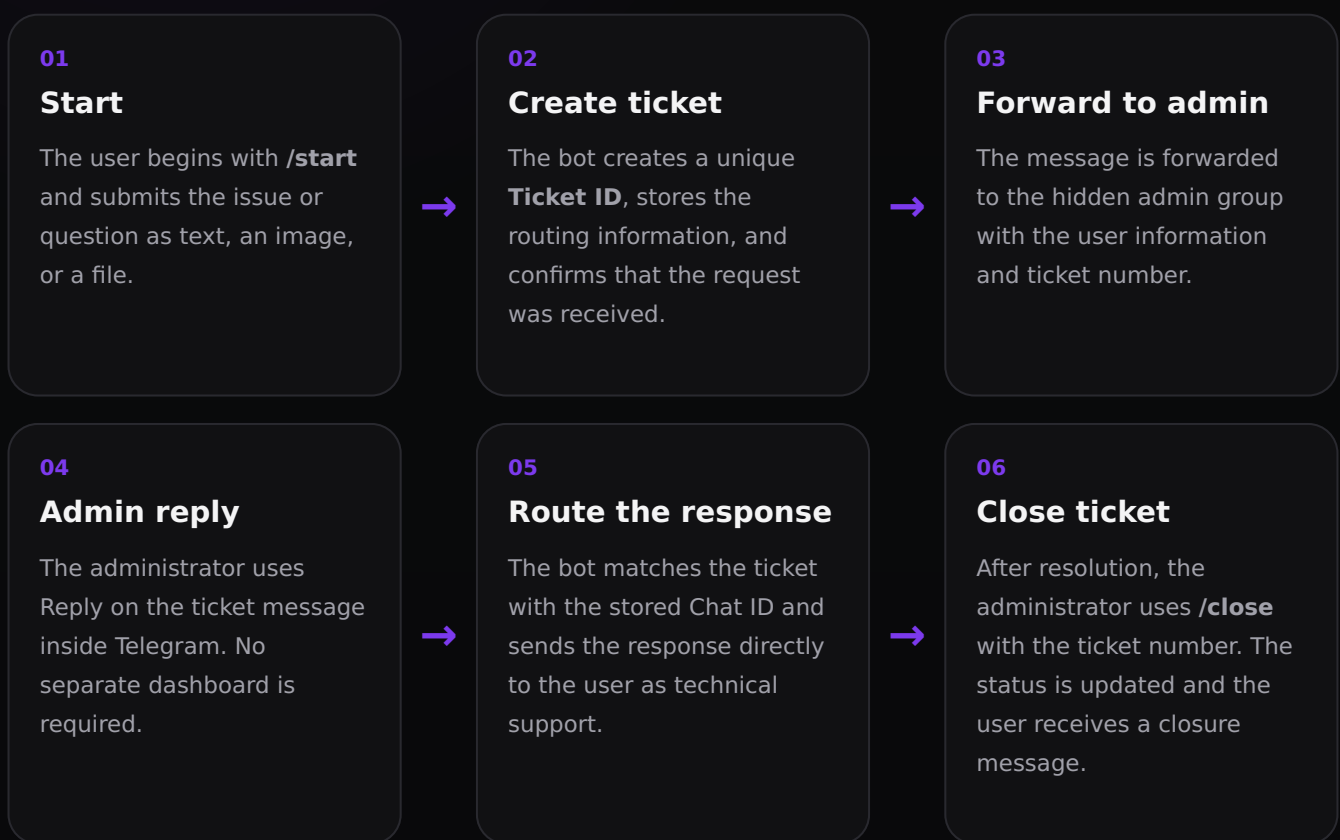
The design does not depend on a full central archive of conversations; it focuses on the minimum data needed for the ticket lifecycle.

Architectural idea: keep the user experience simple inside Telegram while moving ticket-state and routing logic into a lightweight, explicit backend layer.

TICKET LIFECYCLE

From user request to support reply and closure

Telegram remains the simple user-facing interface, while ticket handling happens inside a hidden administration group linked through the ticket identifier and the user's Chat ID.



Why this design?

It separates the user experience from administration details. The user sees a simple bot, the support team works in a private group, and the system handles the mapping between both sides.

Minimum routing data

User identifier / Chat ID, Ticket ID, ticket state (open or closed), and the routing information required to send each reply to the correct user.

Operational result

A direct support experience for users with a structured back-office flow that reduces context loss and prevents ticket replies from becoming mixed.

RELIABILITY & OPERATIONS

Challenges, solutions and operating policies



Delayed replies & Timeouts

Connection handling was adjusted and Polling / temporary caching were used where needed to make the message flow more resilient.



Spam protection

Rate Limiting restricts rapid repeated messages, reducing unnecessary load on the service and preventing disruption for administrators.



Reliable message relay

Ticket ID and Chat ID preserve the correct relationship between the user and administrator response while keeping the admin group hidden.

Bot policies

Policy	Implementation
Privacy	Store the minimum routing and ticket-state information needed for operation instead of a complete central archive of message content.
Anti-Spam	Apply Rate Limiting to reduce rapid repeated submissions and unnecessary load.
Timeout handling	Use connection tuning, Polling, and temporary buffering as needed to improve bot stability.
Ticket closure	Mark the ticket as closed and notify the user to prevent further messages from being mixed into a completed request.

Deployment

The project was deployed on Replit for continuous operation, with a small Flask web server as part of the runtime and external monitoring to help keep the service alive.

What the project demonstrates

Turning a practical support need into a structured workflow, solving reliability and spam problems, designing clear ticket-state logic, and managing an AI-assisted software project from architecture through deployment.

FROM NEED TO SYSTEM

Summary

What this project demonstrates

The bot is a supporting component of the PenGuide ecosystem. It shows how a practical user need - contacting support - can be translated into a manageable technical system with a defined lifecycle and operating policies.

System Architecture

Turning requirements into explicit ticket, routing, reply, and closure logic.

Problem Solving

Handling Timeouts, Spam, and reliable reply routing without exposing the administration group.

AI-Assisted Development

Using Prompt Engineering to guide implementation while keeping system logic and architectural decisions under my direction.

Operations

Managing deployment, hosting, and service monitoring for continuous availability.

PenGuide Support Bot

A standalone portfolio project that remains directly connected to PenGuide as its supporting technical support layer.

Abdullah Nasser AIMsan · System Architecture & AI-Assisted Development